14강

프로그래밍 언어론

서브프로그램 구현

컴퓨터과학과 김진욱교수

학습목차



1. 서브프로그램 구현 개요
2. 정적 체인과 동적 체인
3. 기타 서브프로그램 구현 방법

01

서브프로그램 구현 개요

1. 서브프로그램 구현 개요

❑ 서브프로그램 연결

서브프로그램 호출 시 해야 하는 작업

- 호출자의 상태 저장
- 인수 전달
- 복귀 주소 저장
- 피호출자로 분기

서브프로그램 복귀 시 해야 하는 작업

- 형식인수 값 복사 (출력/입출력 모드인 경우)
- 반환값 전달(함수의 경우)
- 호출자의 상태 복귀
- 호출자로 복귀

1. 서브프로그램 구현 개요

활성 레코드

- 수행 중인 서브프로그램의 데이터 부분이 저장되는 형태
  - 호출자의 상태 정보, 인수, 반환값(함수의 경우), 복귀할 주소 등

활성 레코드(activation record)

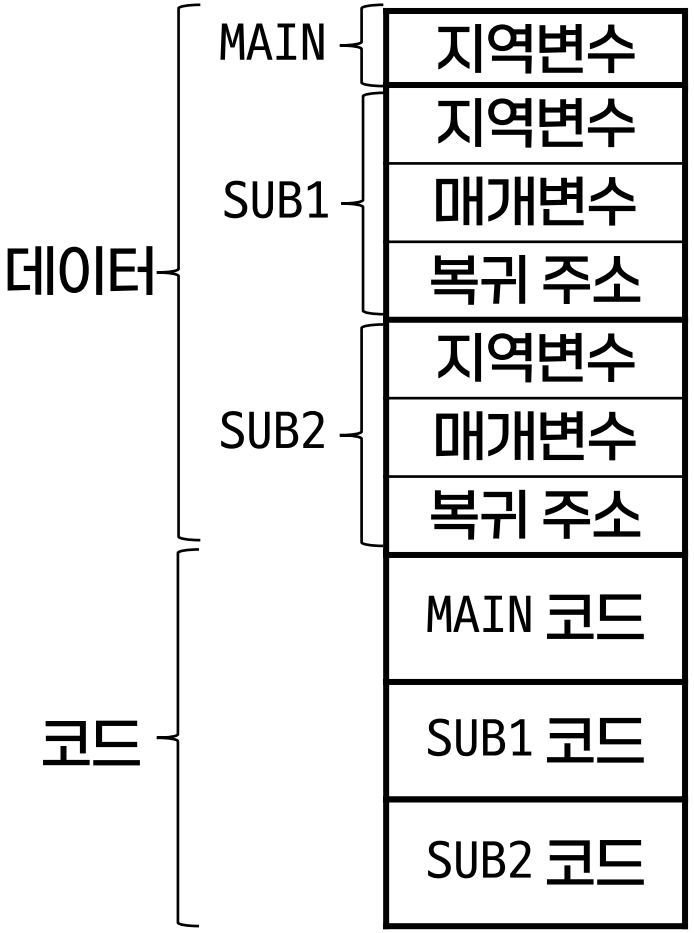
- 저장해야 할 정보의 틀
- 활성 레코드 틀 자체는 정적으로 결정됨

활성 레코드 인스턴스(instance)

- 활성 레코드에 구체적인 값이 들어가 있는 형태
- 저장되는 내용은 동적으로 결정됨

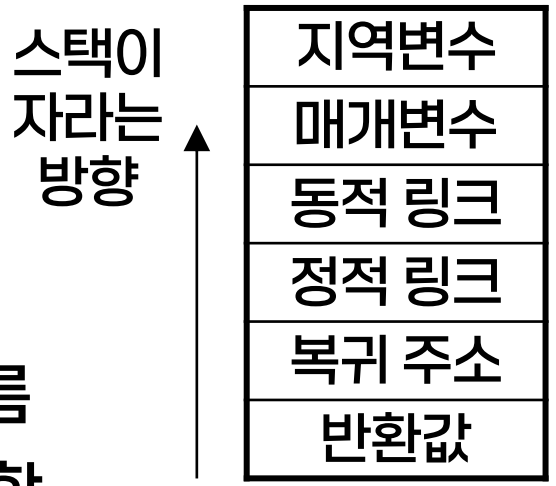
1. 서브프로그램 구현 개요

- Fortran 77의 활성 레코드 구현
  - 활성 레코드를 정적 할당
    - 코드 부분과 데이터 부분을 구별하지 않아도 됨
  - 재귀호출을 허용하지 않음
    - 이유: 재귀호출 시 이전 활성 레코드를 덮어쓰게 되므로 이전 호출 정보를 잃어버리게 됨



1. 서브프로그램 구현 개요

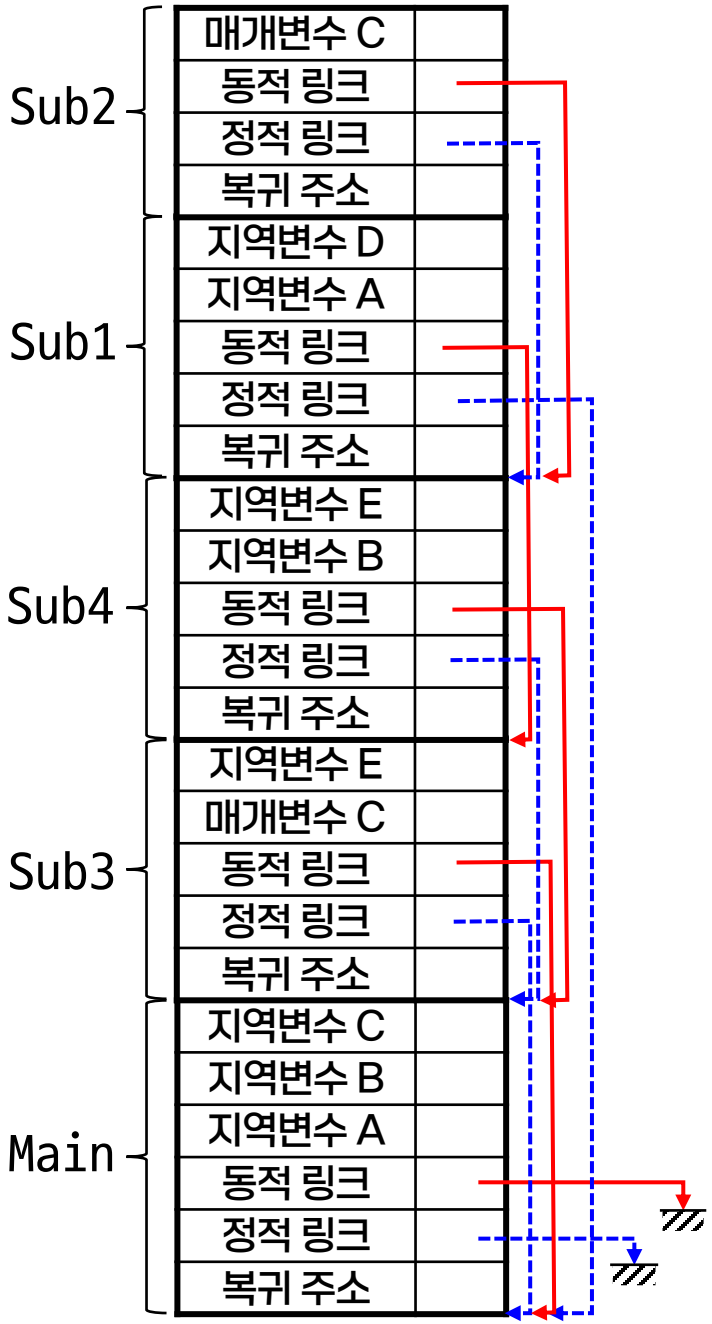
- Algol 형태 언어의 활성 레코드 구현
  - 활성 레코드를 동적 할당
    - 스택에 할당
  - 활성 레코드
    - 형태는 같지만 크기는 서브프로그램마다 다름
    - 서브프로그램 호출 시 동적으로 생성되어야 함
    - 활성 레코드 필드는 프레임 포인터와 변위의 조합으로 참조
  - 프레임 포인터: 활성 레코드의 기준 지점
  - 변위: 프레임 포인터로부터 떨어진 정도
  - 정적 링크: 정적 부모의 프레임 포인터
  - 동적 링크: 호출자의 프레임 포인터



1. 서브프로그램 구현 개요

- Pascal의 활성 레코드 구현

```
program Main;
var
  A, B, C: Integer;
  procedure Sub1;
  var
    A, D: Integer;
    procedure Sub2(C: Integer);
    begin {Sub2}
      A := B + C
    end;
  begin {Sub1}
    Sub2(D);
    B := A + C
  end;
  procedure Sub3(C: Integer);
  var
    E: Integer;
    procedure Sub4;
    var
      B, E: Integer;
    begin {Sub4}
      Sub1;
      E := A + B
    end;
  begin {Sub3}
    Sub4;
    A := C + E
  end;
Begin {Main}
  Sub3(B)
end.
```



1. 서브프로그램 구현 개요

C 언어의 활성 레코드 구현

- 서브프로그램 중첩을 허용하지 않음
  - 정적 링크가 필요 없음
- C 언어의 서브프로그램은 함수이므로 반환값 필요

```
int fac(int n) {
    if (n == 0)
        return 1;
    else
        return n * fac(n-1);
}
int main() {
    int value;
    value = fac(2);
    return 0;
}
```

지역변수
매개변수
동적 링크
복귀 주소
반환값

1. 서브프로그램 구현 개요

❑ 서브프로그램 관련 용어

동적 체인(dynamic chain)

- 활성 레코드 스택상에서 인접한 동적 링크들을 연결한 것
- 리스트 형태

정적 체인(static chain)

- 활성 레코드 스택상에서 정적 링크들을 연결한 것
- 트리 형태

지역 변위(local offset)

- 프레임 포인터에서 스택 동적 변수 위치까지의 변위

02

정적 체인과 동적 체인

2. 정적 체인과 동적 체인

비지역변수 참조

- 참조되는 비지역변수는 활성 레코드 스택상에 존재
- 올바른 활성 레코드만 찾으면
  지역 변위로 변수 위치를 찾을 수 있음

- 정적 영역 규칙: 정적 체인 활용
- 동적 영역 규칙: 동적 체인 활용
  - 참조할 비지역변수를 컴파일 시간에 결정할 수 없음

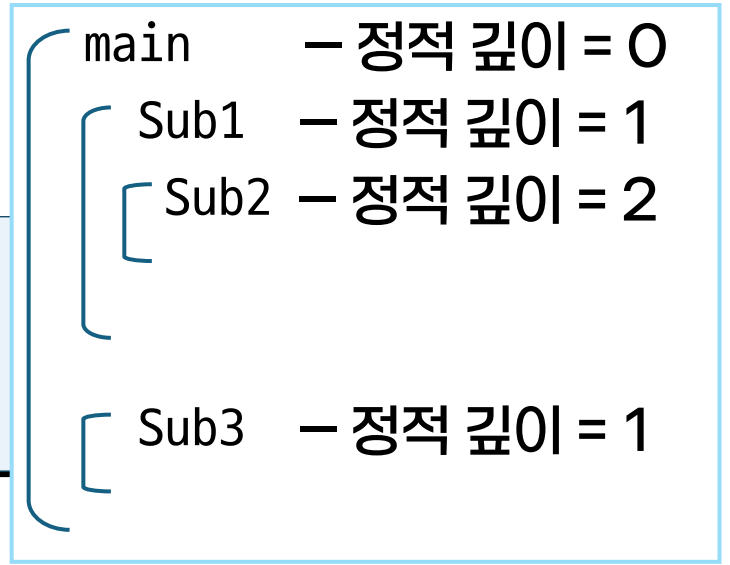
2. 정적 체인과 동적 체인

정적 체인

- 변수 참조: (체인 변위, 지역 변위)

정적 깊이(static depth)

- 정적 영역의 중첩 깊이
- 자신을 감싸고 있는 정적 영역의 개수



체인 변위(chain offset)

- 비지역변수를 참조하는 정적 영역의 정적 깊이와 비지역변수가 선언된 정적 영역의 정적 깊이의 차이

- 체인 변위가 커질수록 변수 참조 비용도 커짐

2. 정적 체인과 동적 체인

정적 체인

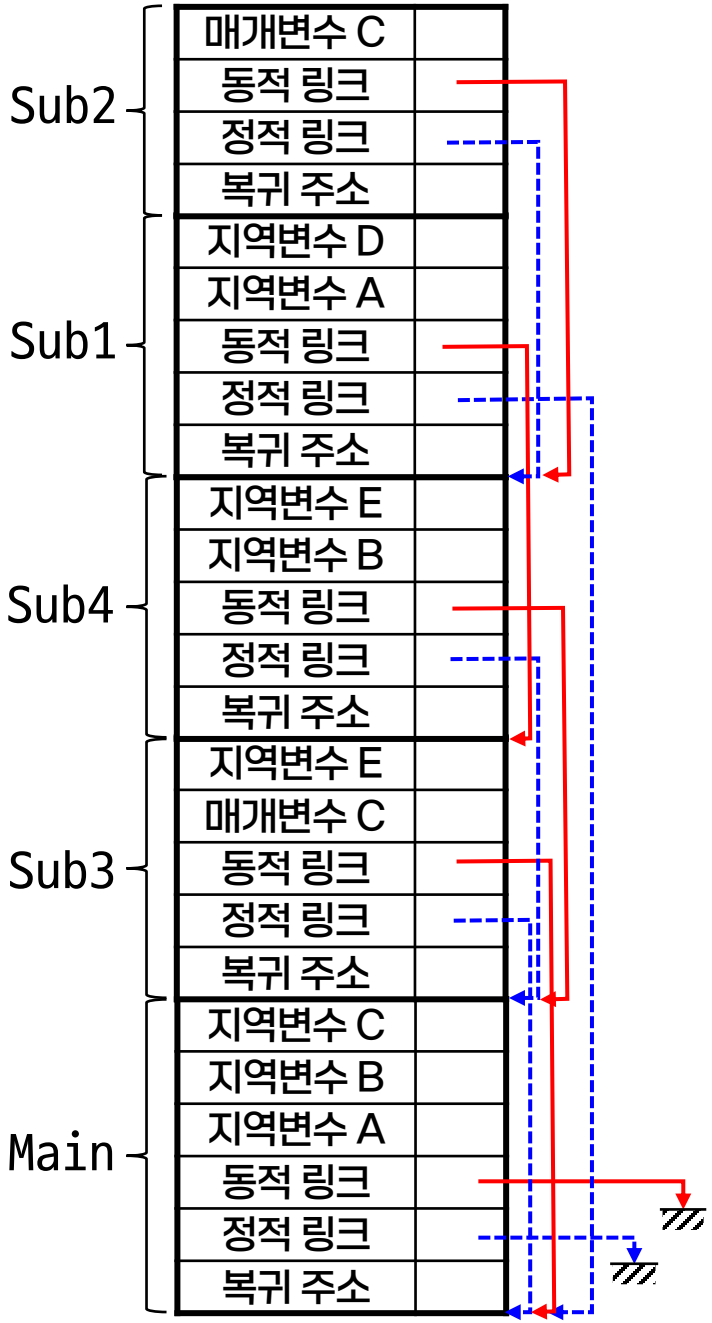
```
program Main;
var
  A, B, C: Integer;
  procedure Sub1;
  var
    A, D: Integer;
    procedure Sub2(C: Integer);
    begin {Sub2}
      A := B + C
    end;
  begin {Sub1}
    Sub2(D);
    B := A + C
  end;
  procedure Sub3(C: Integer);
  var
    E: Integer;
    procedure Sub4;
    var
      B, E: Integer;
    begin {Sub4}
      Sub1;
      E := A + B
    end;
  begin {Sub3}
    Sub4;
    A := D + E
  end;
Begin {Main}
  Sub3(B)
end.
```

```
A (1, 3)
B (2, 4)
C (0, 3)
```

```
B (1, 4)
A (0, 3)
C (1, 5)
```

```
E (0, 4)
A (2, 3)
B (0, 3)
```

```
A (1, 3)
D 오류
E (0, 4)
```



2. 정적 체인과 동적 체인

정적 체인 구성

- 서브프로그램 호출 시 정적 링크는 정적 부모의 가장 최근 활성 레코드를 가리키도록 설정
- 정적 부모의 최근 활성 레코드 탐색 방법
  - 방법 1: 동적 링크를 계속 거슬러 올라감(비효율적)
  - 방법 2: 호출자와 피호출자의 정적 깊이 차이를 이용

```
main      – 정적 깊이 = 0
  Sub1    – 정적 깊이 = 1
    Sub2  – 정적 깊이 = 2

  Sub3    – 정적 깊이 = 1
    Sub4  – 정적 깊이 = 2
```

2. 정적 체인과 동적 체인

- 정적 체인 방법의 평가
  - 장점
    - 구현이 쉬움
    - 공간 낭비가 적음
  - 단점
    - 체인 변위가 큰 경우 비지역변수 참조 시간 부담이 커짐
    - 비지역변수 참조 시간이 변수에 따라 다름

2. 정적 체인과 동적 체인

- 동적 체인
  - 변수 참조
    - 비지역변수를 찾을 때까지 동적 링크를 거슬러 올라감
  - 호출 순서에 따라 참조하는 비지역변수가 달라질 수 있음
  - 활성 레코드에 변수 이름을 저장해야 함(비효율적)

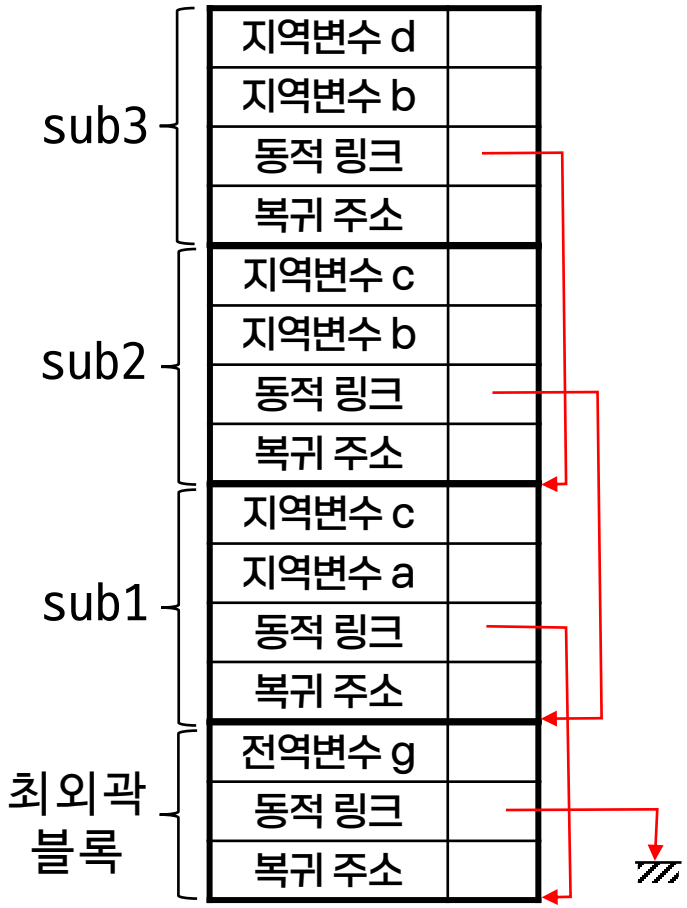
깊은 참조(deep access)

- 동적 체인을 따라가며 변수를 찾는 참조 방법

2. 정적 체인과 동적 체인

❑ 동적 체인

```
sub sub3 {
  local $b=32, $d=34;
  $a = $b + $c;
}
sub sub2 {
  local $b=22, $c=23;
  &sub3;
}
sub sub1() {
  local $a=11, $c=13;
  &sub2;
}
$g = 0;
&sub1;
```



03

기타 서브프로그램 구현 방법

3. 기타 서브프로그램 구현 방법

디스플레이

- 정적 체인 방법을 대체할 수 있는 방법
- 수행 중 참조할 수 있는 모든 정적 링크를 별도의 스택(디스플레이)에 관리

디스플레이(display)

- 현재 수행 중인 블록과 이 블록의 모든 정적 조상에 대한 활성 레코드를 가리키는 포인터로 이루어진 스택

3. 기타 서브프로그램 구현 방법

- 디스플레이
  - 변수 참조: (디스플레이 변위, 지역 변위)
  - 디스플레이 변위(display offset)
    - 원하는 활성 레코드에 대한 포인터가 디스플레이 스택에 저장된 위치를 나타내는 색인 값
    - 찾을 변수가 선언된 서브프로그램의 정적 깊이와 동일
  - 변수 참조 방법
    - 디스플레이 변위를 이용하여 참조할 변수를 포함하는 활성 레코드 선택
    - 지역 변위를 이용하여 해당 활성 레코드상에서 변수 위치 선택
  - 현재 수행 중인 서브프로그램의 정적 깊이가 p라면 디스플레이 내에는 p+1개의 항목 존재

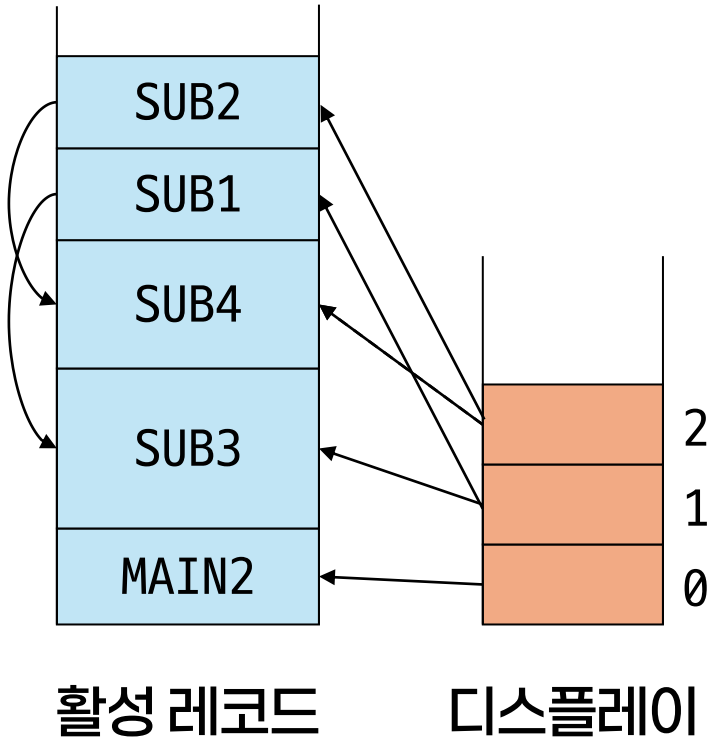
3. 기타 서브프로그램 구현 방법

- 디스플레이 관리
  - 피호출자의 정적 깊이가 q인 경우
  - 서브프로그램 호출 시
    - 디스플레이 D의 q번째 항목 D[q]를 피호출자의 활성 레코드에 백업 후
    - 새로 생성된 활성 레코드에 대한 포인터를 D[q]에 저장
  - 서브프로그램 복귀 시
    - 현재 활성 레코드 내에 백업되었던 D[q] 항목을 복구

3. 기타 서브프로그램 구현 방법

- 디스플레이 관리

```
program MAIN2;
  procedure SUB1;
    procedure SUB2;
    begin
    end; {SUB2}
  begin
    SUB2;
  end; {SUB1}
  procedure SUB3;
    procedure SUB4;
    begin
      SUB1;
    end; {SUB4}
  begin
    SUB4;
  end; {SUB3}
Begin
  SUB3;
end. {MAIN2}
```



3. 기타 서브프로그램 구현 방법

정적 체인과 디스플레이 비교

	정적 체인	디스플레이
지역변수 참조	두 방법 모두 비슷한 비용	
비지역변수 참조	한 단계 떨어진 경우, 디스플레이와 같은 비용	멀리 떨어져 있는 경우에 유리함
서브프로그램 호출	정적 깊이 차가 적으면 유리	정적 깊이 차가 크면 유리
서브프로그램 복귀	상수 시간 (약간 빠름)	상수 시간 (백업 복구 필요)

3. 기타 서브프로그램 구현 방법

변수 스택과 중앙 테이블

- 동적 체인 방법의 비효율성을 개선할 수 있는 방법

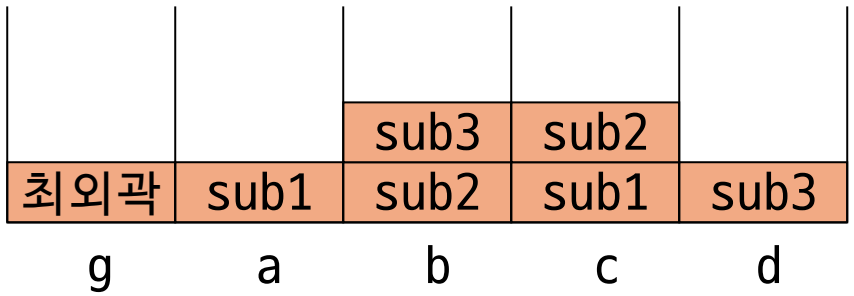
얕은 참조(shallow access)

- 비지역변수와 지역변수를 구별하지 않음
- 참조되는 모든 변수의 정보는 하나의 자료 구조 내에 존재
- 변수 스택 방법, 중앙 테이블 방법

3. 기타 서브프로그램 구현 방법

변수 스택

- 변수 이름마다 별도의 스택을 두는 방법
- 스택에는 서브프로그램의 활성 레코드에 대한 포인터가 저장됨

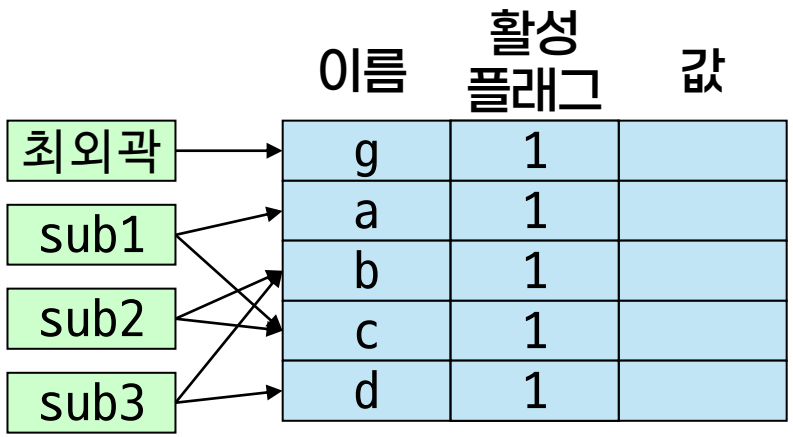


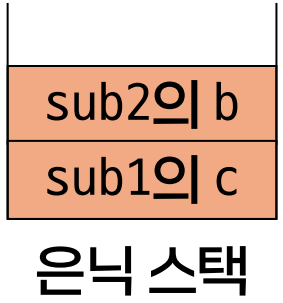

```
sub sub3 {
  local $b=32, $d=34;
  $a = $b + $c;
}
sub sub2 {
  local $b=22, $c=23;
  &sub3;
}
sub sub1() {
  local $a=11, $c=13;
  &sub2;
}
$g = 0;
&sub1;
```

3. 기타 서브프로그램 구현 방법

중앙 테이블

- 각 변수에 대해 실제로 사용할 수 있는지 여부와 실제 값을 하나의 표에 저장해 두는 방법
- 변수의 활성 플래그가 이미 활성화되어 있으면 은닉 스택에 백업





은닉 스택

```
sub sub3 {
  local $b=32, $d=34;
  $a = $b + $c;
}
sub sub2 {
  local $b=22, $c=23;
  &sub3;
}
sub sub1() {
  local $a=11, $c=13;
  &sub2;
}
$g = 0;
&sub1;
```

3. 기타 서브프로그램 구현 방법

- 과거 민감(history-sensitive) 서브프로그램
  - 이전 호출 내용을 기억하는 서브프로그램
    - 서브프로그램이 종료되어도 상태 정보 일부가 기억됨
  - 과거 민감 서브프로그램 구현
    - 전역변수 이용: 정보 은닉이 보장되지 않음
    - 정적 지역변수 이용: 정보 은닉 보장

3. 기타 서브프로그램 구현 방법

- 코루틴(coroutine)
  - 여러 개의 진입 위치를 스스로 관리하는 서브프로그램
  - 과거 중단 위치를 기억해 두었다가 다음에 개재됨(resumed)
    - 활성 레코드가 삭제되지 않고 유지되어야 함
  - 서브프로그램 호출 관계가 대등(대칭적 제어 모델)
  - 여러 코루틴이 번갈아 수행됨(유사병행성)
  - 지원 언어
    - Simula 67, BLISS, InterLISP, Modula-2, Python
  - 시뮬레이션 분야에 유용
    - 생산자-소비자 시뮬레이션, 카드 게임 시뮬레이션 등

정리하기

- 서브프로그램 호출과 복귀를 합쳐 서브프로그램 연결이라고 부르며
  이때 제어 흐름뿐만 아니라 자료 흐름도 관리해야 함
- 자료 흐름 관리에 사용하는 활성 레코드에는 서브프로그램의 데이터뿐 아니라
  호출자의 상태, 복귀 주소, 반환값, 각종 관리 데이터가 저장됨
- 활성 레코드에는 영역 규칙을 구현하기 위해 동적 링크와 정적 링크가 사용됨
- 정적 링크가 연결된 정적 체인은 정적 영역 규칙을 구현하기 위해 사용되고,
  동적 링크가 연결된 동적 체인은 동적 영역 규칙을 구현하기 위해 사용됨
- 정적 체인을 거슬러 올라가는 부담을 줄이기 위해서 디스플레이를 활용
- 동적 영역 규칙에서 모든 변수에 대한 정보를 한곳에서 관리하는 방법인
  얕은 참조에는 변수 스택과 중앙 테이블이 있음
- 과거 민감 서브프로그램은 이전의 호출 내용을 기억하는 서브프로그램임
- 코루틴은 여러 개의 진입 지점을 스스로 관리하는 서브프로그램임

15강

다음시간안내 ▶▶▶

객체지향과 다형성